

Homework 12: Tumor Phylogenetics

Bernhard Buss Dario Franzen Aurimas Greicius
Timothy Lauren Nijhuis Carlo Rossetto Yuliyana Slavchev
Görkem Kadir Solun

```
set.seed(42)
```

Problem 29: Diploid genotype error model

Calculate the following probabilities analogously:

1. $W = TT$, and $Y = AG$

$$\begin{aligned} P(TT \mid AG) &= \frac{\delta}{2} \frac{\epsilon}{2} \frac{1}{3} && \text{dropout with sequencing error } \textcolor{red}{A}\textcolor{black}{G} \rightarrow TT \\ &+ \frac{\delta}{2} \frac{\epsilon}{2} \frac{1}{3} && \text{dropout with sequencing error } \textcolor{black}{A}\textcolor{red}{G} \rightarrow TT \\ &= \frac{\delta\epsilon}{6} \end{aligned}$$

2. $W = AT$, and $Y = AG$

$$\begin{aligned} P(AT \mid AG) &= (1 - \delta) \frac{\epsilon}{2} \frac{1}{3} && \text{no dropout, sequencing error } \textcolor{red}{A}\textcolor{black}{G} \rightarrow AT \\ &= \frac{\epsilon}{6} - \frac{\delta\epsilon}{6} \end{aligned}$$

3. $W = AG$, and $Y = AG$

$$\begin{aligned} P(AG \mid AG) &= (1 - \delta)(1 - \epsilon) && \text{no dropout without sequencing error } AG \rightarrow AG \\ &= 1 - \delta - \epsilon + \delta\epsilon \end{aligned}$$

4. $W = AA$, and $Y = AA$

$$\begin{aligned}
P(AA \mid AA) &= (1 - \delta)(1 - \epsilon) && \text{no dropout without sequencing error } AA \rightarrow AA \\
&+ \frac{\delta}{2}(1 - \frac{\epsilon}{2}) && \text{dropout without sequencing error } \cancel{A}A \rightarrow AA \\
&+ \frac{\delta}{2}(1 - \frac{\epsilon}{2}) && \text{dropout without sequencing error } A\cancel{A} \rightarrow AA \\
&= 1 - \epsilon + \frac{\delta\epsilon}{2}
\end{aligned}$$

5. $W = AG$, and $Y = AA$

$$\begin{aligned}
P(AG \mid AA) &= (1 - \delta)\frac{\epsilon}{2}\frac{1}{3} && \text{no dropout, sequencing error } A\textcolor{red}{A} \rightarrow AG \\
&+ (1 - \delta)\frac{\epsilon}{2}\frac{1}{3} && \text{no dropout, sequencing error } \textcolor{red}{A}A \rightarrow AG \\
&= \frac{\epsilon}{3} - \frac{\delta\epsilon}{3}
\end{aligned}$$

6. $W = GG$, and $Y = AA$

$$\begin{aligned}
P(GG \mid AA) &= \frac{\delta}{2}\frac{\epsilon}{2}\frac{1}{3} && \text{dropout with sequencing error } \textcolor{red}{A}\cancel{A} \rightarrow GG \\
&+ \frac{\delta}{2}\frac{\epsilon}{2}\frac{1}{3} && \text{dropout with sequencing error } \cancel{A}\textcolor{red}{A} \rightarrow GG \\
&= \frac{\delta\epsilon}{6}
\end{aligned}$$

Problem 30: The Beta-Binomial distribution

For a binomial random variable $X \sim \text{Binom}(n, p)$,

$$\mathbb{E}[X] = np, \quad \text{Var}(X) = np(1 - p).$$

For a beta-binomial random variable $X \sim \text{BetaBinom}(n, \alpha, \beta)$,

$$\mathbb{E}[X] = n \frac{\alpha}{\alpha + \beta},$$

and

$$\text{Var}(X) = n \frac{\alpha\beta}{(\alpha + \beta)^2} \frac{\alpha + \beta + n}{\alpha + \beta + 1}.$$

In all four requested cases, the distribution is symmetric around 500, so the mean is 500. The variances differ strongly because the beta binomial contains extra variability from the random success probability.

```
n_trials <- 1000L
n_experiments <- 2000L

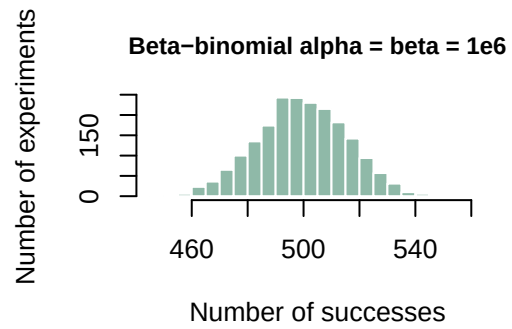
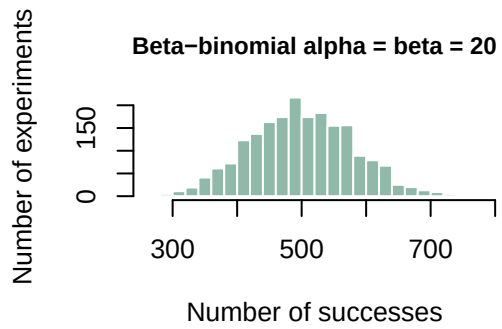
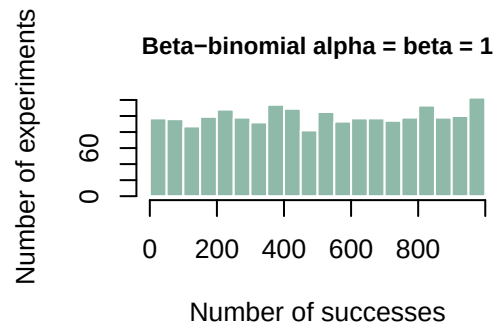
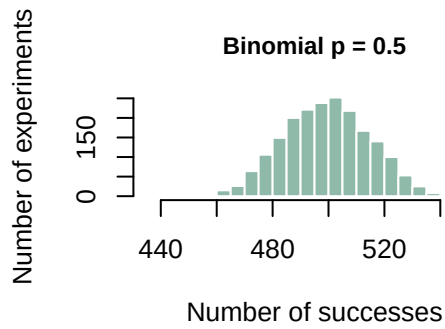
theoretical_summary <- data.frame(
  Distribution = c(
    "Binomial, p = 0.5",
    "Beta-binomial, alpha = beta = 1",
    "Beta-binomial, alpha = beta = 20",
    "Beta-binomial, alpha = beta = 1e6"
  ),
  Mean = c(
    n_trials * 0.5,
    n_trials * 0.5,
    n_trials * 0.5,
    n_trials * 0.5
  ),
  Variance = c(
    n_trials * 0.5 * 0.5,
    n_trials * (2 * 1 + n_trials) / (4 * (2 * 1 + 1)),
    n_trials * (2 * 20 + n_trials) / (4 * (2 * 20 + 1)),
    n_trials * (2 * 1e6 + n_trials) / (4 * (2 * 1e6 + 1))
  )
)
```

```
)
```

```
knitr::kable(theoretical_summary, digits = 3)
```

Distribution	Mean	Variance
Binomial, $p = 0.5$	500	250.000
Beta-binomial, $\alpha = \beta = 1$	500	83500.000
Beta-binomial, $\alpha = \beta = 20$	500	6341.463
Beta-binomial, $\alpha = \beta = 1e6$	500	250.125

```
rbeta_binom <- function(n, size, alpha, beta) {  
  p <- rbeta(n, shape1 = alpha, shape2 = beta)  
  rbinom(n, size = size, prob = p)  
}  
  
samples <- list(  
  "Binomial p = 0.5" =  
    rbinom(n_experiments, size = n_trials, prob = 0.5),  
  "Beta-binomial alpha = beta = 1" =  
    rbeta_binom(n_experiments, size = n_trials, alpha = 1, beta = 1),  
  "Beta-binomial alpha = beta = 20" =  
    rbeta_binom(n_experiments, size = n_trials, alpha = 20, beta = 20),  
  "Beta-binomial alpha = beta = 1e6" =  
    rbeta_binom(n_experiments, size = n_trials, alpha = 1e6, beta = 1e6)  
)  
  
old_par <- par(mfrow = c(2, 2), mar = c(4.2, 4.2, 3, 1))  
for (nm in names(samples)) {  
  hist(  
    samples[[nm]],  
    breaks = 30,  
    main = nm,  
    xlab = "Number of successes",  
    ylab = "Number of experiments",  
    col = "#8FB9A8",  
    border = "white",  
    cex.main = 0.9  
  )  
}
```



```
par(old_par)
```

Problem 31: Likelihood of binary genotypes given a tumor mutation tree

Part 1

For an attachment of cell j to node v , the expected genotype has mutation i present exactly when mutation i lies on the path from the root to v . If x_{ij} is the observed binary genotype, then

$$P(x_{ij} \mid z_i = 0) = \alpha^{x_{ij}}(1 - \alpha)^{1-x_{ij}},$$

and

$$P(x_{ij} \mid z_i = 1) = (1 - \beta)^{x_{ij}}\beta^{1-x_{ij}},$$

where α is the false positive rate and β is the false negative rate.

For each cell, start with the log likelihood under the all absent genotype. When traversing the tree from parent to child, add only the log-likelihood increment for the mutation at the child node. Thus every mutation cell pair is processed a constant number of times, giving $O(IJ)$ time.

```
##' computes the likelihood of the input genotype matrix given a tree
##' returns both the node-by-cell log likelihood, and the overall log likelihood
##' after marginalizing over all cell attachment
##' assume uniform attachment probabilities
##' @param tree list with $root (integer node index) and $children
##' (a list mapping each parent node index, as a string, to an integer vector
##' of its children's indices,
##' e.g. list(root=1L, children=list("1"=2L, "2"=c(3L,4L))))
##' @param genotype_matrix mutation-by-cell binary matrix of observed genotypes
##' @param alpha false positive rate
##' @param beta false negative rate
compute_log_lik <- function(tree, genotype_matrix, alpha, beta) {

  num_mutations <- nrow(genotype_matrix)
  num_cells <- ncol(genotype_matrix)

  log_lik <- matrix(
    NA_real_,
    nrow = num_mutations,
    ncol = num_cells,
    dimnames = list(
```

```

    paste0("node_", seq_len(num_mutations)),
    paste0("cell_", seq_len(num_cells))
  )
)

log_absent <- ifelse(genotype_matrix == 1, log(alpha), log1p(-alpha))
log_present <- ifelse(genotype_matrix == 1, log1p(-beta), log(beta))

absent_baseline <- colSums(log_absent)
present_increment <- log_present - log_absent

stack <- tree$root
log_lik[tree$root, ] <- absent_baseline + present_increment[tree$root, ]

while (length(stack) > 0) {
  parent <- stack[[length(stack)]]
  stack <- stack[-length(stack)]

  children <- tree$children[[as.character(parent)]]
  if (is.null(children)) {
    next
  }

  for (child in children) {
    log_lik[child, ] <- log_lik[parent, ] + present_increment[child, ]
    stack <- c(stack, child)
  }
}

if (anyNA(log_lik)) {
  stop("The tree does not reach every mutation node in the genotype matrix.")
}

cell_log_lik <- vapply(seq_len(num_cells), function(cell_idx) {
  node_log_lik <- log_lik[, cell_idx]
  max_log_lik <- max(node_log_lik)
  max_log_lik + log(mean(exp(node_log_lik - max_log_lik)))
}, numeric(1))

final_llik <- sum(cell_log_lik)

return(list(log_lik = log_lik, final_llik = final_llik))

```

```
}
```

Part 2

```
genotype_matrix <- as.matrix(read.table("cells.txt"))
colnames(genotype_matrix) <- paste0("cell_", seq_len(ncol(genotype_matrix)))
rownames(genotype_matrix) <- paste0("mut_", seq_len(nrow(genotype_matrix)))
genotype_matrix
```

	cell_1	cell_2	cell_3	cell_4	cell_5	cell_6	cell_7	cell_8
mut_1	1	1	1	1	1	1	1	1
mut_2	1	1	0	1	1	1	1	1
mut_3	1	1	1	0	0	0	0	0
mut_4	0	0	0	0	0	1	1	1
mut_5	0	0	0	0	1	1	0	0

The provided file contains 5 mutations and 8 cells. The problem statement mentions 7 cells, but the matrix in cells.txt has 8 columns. We use the data as given. This matrix does not permit a perfect phylogeny without errors. In a perfect phylogeny under the infinite sites assumption, for any pair of mutations, the sets of cells carrying those mutations must be nested or disjoint. Here, for example,

$$S_2 = \{1, 2, 4, 5, 6, 7, 8\},$$
$$S_3 = \{1, 2, 3\}.$$

These two sets overlap in cells $\{1, 2\}$, but neither set contains the other. Similarly, $S_4 = \{6, 7, 8\}$ and $S_5 = \{5, 6\}$ overlap without being nested. Such crossing mutation patterns cannot be represented by a perfect phylogeny with each mutation occurring exactly once and never being lost.

Parts 3 and 4

Use the following mutation trees.

```

tree1 <- list(
  root      = 1L,
  children = list(
    "1" = 2L,
    "2" = c(3L, 4L),
    "4" = 5L
  )
)

tree2 <- list(
  root      = 1L,
  children = list(
    "1" = 2L,
    "2" = c(3L, 4L, 5L)
  )
)

```

```

cases <- list(
  "T1: alpha=0.1, beta=0.01" =
    list(tree = tree1, alpha = 0.1, beta = 0.01),
  "T2: alpha=0.1, beta=0.01" =
    list(tree = tree2, alpha = 0.1, beta = 0.01),
  "T1: alpha=0.01, beta=0.1" =
    list(tree = tree1, alpha = 0.01, beta = 0.1),
  "T2: alpha=0.01, beta=0.1" =
    list(tree = tree2, alpha = 0.01, beta = 0.1)
)

```

```

results <- lapply(cases, function(case) {
  compute_log_lik(
    tree = case$tree,
    genotype_matrix = genotype_matrix,
    alpha = case$alpha,
    beta = case$beta
  )
})

```

```

best_attachment <- lapply(results, function(result) {
  apply(result$log_lik, 2, which.max)
})

```

```

result_table <- do.call(rbind, lapply(names(results), function(nm) {

```

```

data.frame(
  Case = nm,
  Log_likelihood = results[[nm]]$final_llik,
  Best_node_by_cell = paste(best_attachment[[nm]], collapse = ", ")
)
}))
row.names(result_table) <- NULL

knitr::kable(result_table, digits = 3)

```

Case	Log_likelihood	Best_node_by_cell
T1: alpha=0.1, beta=0.01	-18.490	3, 3, 1, 2, 2, 5, 4, 4
T2: alpha=0.1, beta=0.01	-17.957	3, 3, 1, 2, 5, 4, 4, 4
T1: alpha=0.01, beta=0.1	-19.362	3, 3, 3, 2, 5, 5, 4, 4
T2: alpha=0.01, beta=0.1	-21.106	3, 3, 3, 2, 5, 4, 4, 4

The heatmaps below show the node-by-cell log likelihoods returned by `compute_log_lik`. Larger values indicate more likely posthoc attachment of the corresponding cell to that node.

```

plot_log_lik_heatmap <- function(log_lik, main) {
  num_nodes <- nrow(log_lik)
  num_cells <- ncol(log_lik)
  z <- t(log_lik[num_nodes:1, , drop = FALSE])

  image(
    x = seq_len(num_cells),
    y = seq_len(num_nodes),
    z = z,
    col = hcl.colors(80, "Viridis"),
    axes = FALSE,
    xlab = "Cells (observed genotypes)",
    ylab = "Node (expected tree profiles)",
    main = main
  )
  axis(1, at = seq_len(num_cells), labels = seq_len(num_cells))
  axis(2, at = seq_len(num_nodes), labels = num_nodes:1, las = 1)
  box()

  text(
    x = rep(seq_len(num_cells), times = num_nodes),

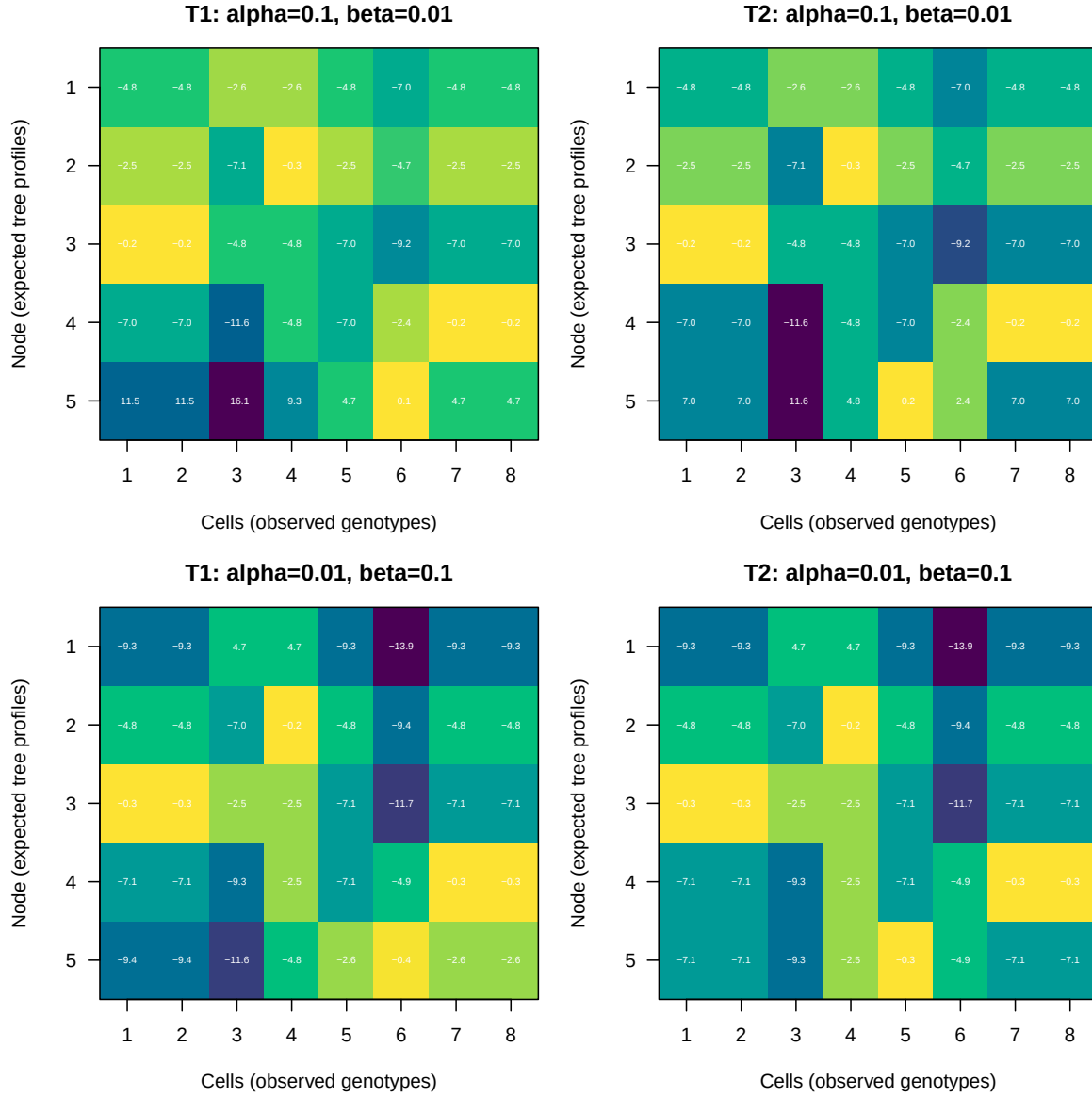
```

```

    y = rep(seq_len(num_nodes), each = num_cells),
    labels = sprintf("%.1f", as.vector(z)),
    cex = 0.55,
    col = "white"
  )
}

old_par <- par(mfrow = c(2, 2), mar = c(4.2, 4.2, 3, 1))
for (nm in names(results)) {
  plot_log_lik_heatmap(results[[nm]]$log_lik, nm)
}

```



`par(old_par)`

When false positives are more common than false negatives (when $\alpha > \beta$), $(\alpha, \beta) = (0.1, 0.01)$, Tree 2 has the higher marginal log likelihood:

$$-17.957 > -18.490.$$

This setting makes it cheap to explain extra observed 1's as false positives. Tree 2 therefore fits the data better because mutation 5 is a sibling of mutations 3 and 4, allowing cell 5 to attach directly to node 5 without forcing mutation 4 to be present.

When false negatives are more common than false positives (when $\beta > \alpha$), $(\alpha, \beta) = (0.01, 0.1)$, Tree 1 has the higher marginal log likelihood:

$$-19.362 > -21.106.$$

This setting makes missing observed 1's less costly, but extra observed 1's now cost $\log \alpha = \log(0.01) \approx -4.6$ each. Tree 1 has the ancestry $4 \rightarrow 5$, so cell 6, which carries both mutations 4 and 5, fits node 5 of Tree 1 perfectly. In Tree 2, mutations 4 and 5 are siblings, so no single node carries both, and cell 6 must incur at least one false positive under either attachment. This penalty is what tips the balance toward Tree 1 when false positives are expensive. Thus the preferred tree and the maximum-likelihood cell attachments change with the error parameters.